# TP N°6

## ASP.NET CORE WEB API

## CLEAN ARCHITECTURE



**Enseignant :** Malek Zribi

# INTRODUCTION

- Clean architecture est une philosophie de conception logicielle qui organise le code en couches distinctes avec des responsabilités claires.
- L'idée clé est de séparer les préoccupations afin que chaque couche de l'application soit indépendante des autres.
- Les modifications ou mises à jour apportées à une partie du système auront un impact minimal, voire nul, sur les autres.

# POURQUOI CLEAN ARCHITECTURE EST IMPORTANTE ?

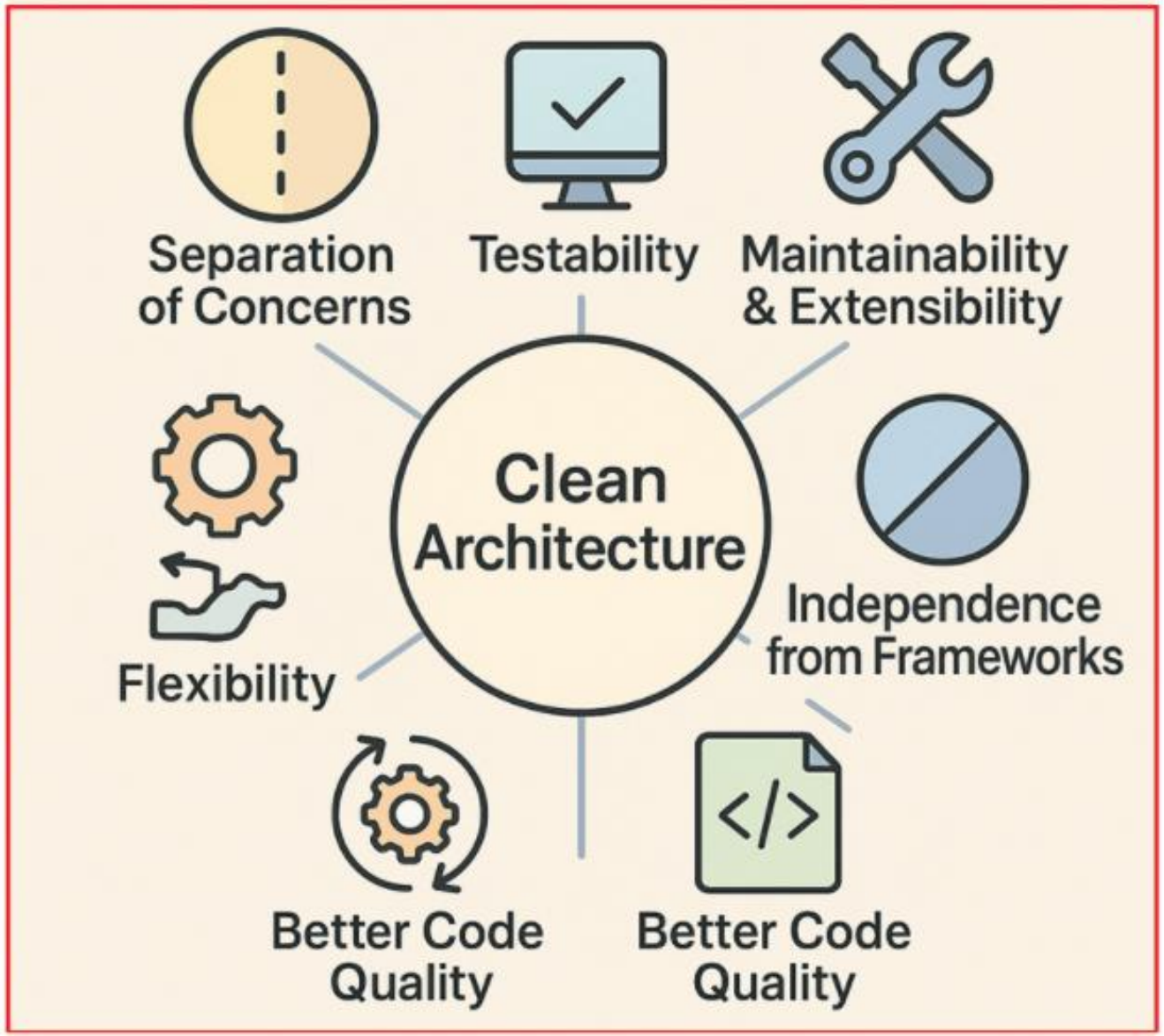



**Tightly coupled:** One failure stops the entire process



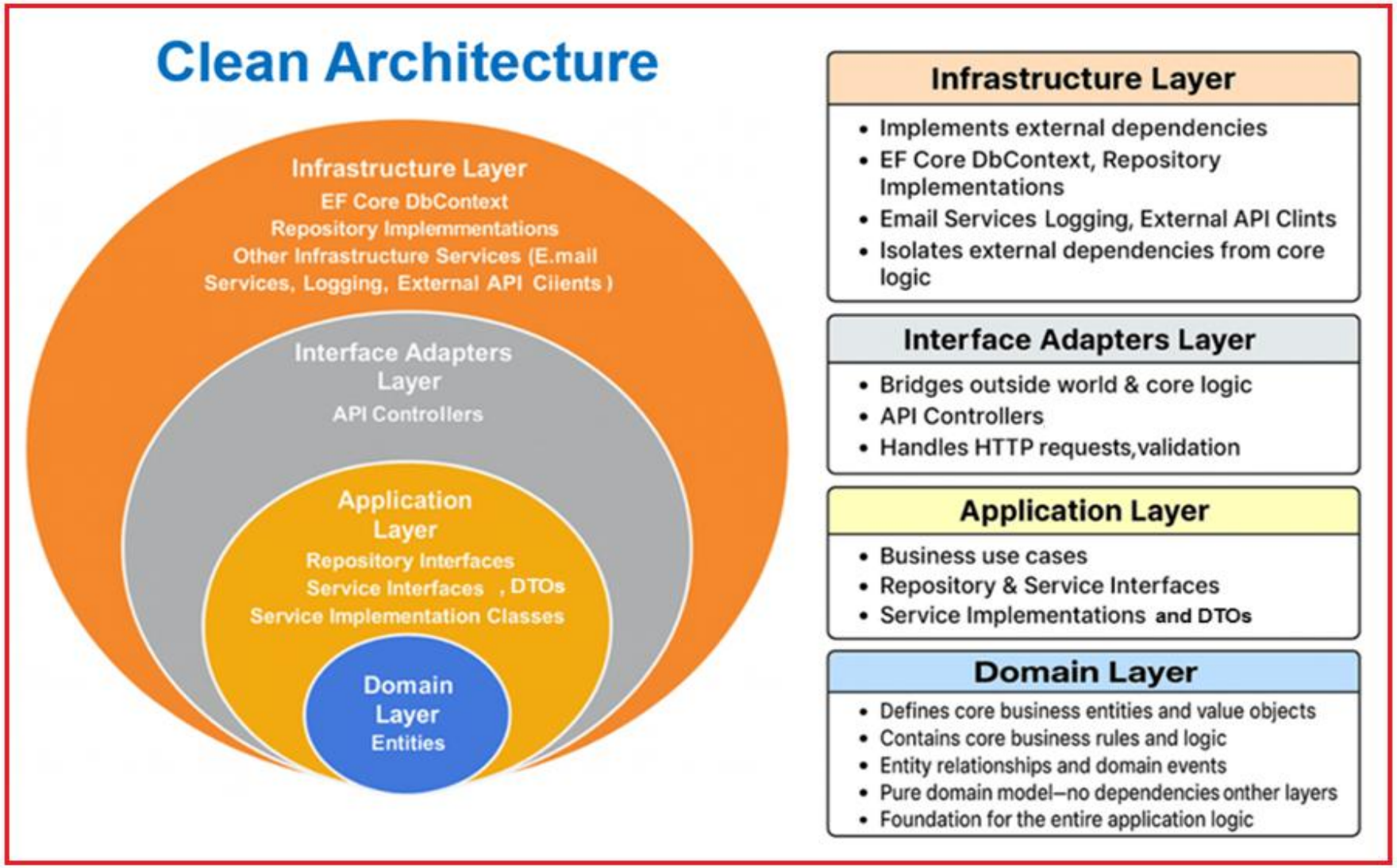

**Clean Architecture:** Each machine operates independently. Failure isolated.

# AVANTAGES DE CLEAN ARCHITECTURE



# COMPRENDRE LES COUCHES DE L'ARCHITECTURE PROPRE

- « Clean architecture » organise le système en quatre couches distinctes.



# COUCHE DE DOMAINE

- La couche de domaine est le cœur de la logique métier, elle comprend un modèle de domaine pur qui contient des règles métier et des validations, indépendamment des préoccupations liées à l'infrastructure et à l'application.

- Contient les Entités : Les objets métier principaux (tels que Produit, Employé, Paiement, Commande, etc.) dotés de propriétés et de règles de validation métier.
- Par exemple, l'entité Produit garantit que le prix est positif et que le stock n'est pas négatif.

# COUCHE D'INFRASTRUCTURE

La couche Infrastructure implémente toutes les dépendances externes, notamment les bases de données, les API externes et les systèmes de fichiers.

Elle fournit les classes concrètes qui implémentent les interfaces définies dans la couche Application. Elle contient :

- EF Core DbContext : la classe utilisée pour interagir avec votre base de données SQL Server (par exemple, ApplicationDbContext), définissant les tables et les relations.
- Implémentations des Repository: classes concrètes qui utilisent EF Core DbContext pour effectuer les opérations CRUD telles que définies par les interfaces Repository dans la couche application.
- Autres services d'infrastructure : services de messagerie électronique, journalisation, clients API externes.

# COUCHE APPLICATION

Elle contient :

- **Interfaces Repository** : Elles définissent le contrat des opérations d'accès aux données requises par la logique métier, sans préciser comment les données sont stockées ou récupérées (cette responsabilité incombe à l'infrastructure).
- **Interfaces de service :** définissent les opérations métier (cas d'utilisation) telles que la création, la récupération, la mise à jour et la suppression de ressources (par exemple, produits, employés).
- **Implémentations de services :** Elles mettent en œuvre des interfaces de service qui encapsulent les flux de travail métier. Elles implémentent la logique métier en appelant des référentiels et en gérant les entités du domaine, souvent en prenant en charge la validation et les règles métier.
- **Les DTO (Data Transfer Objects) :** Définissent les structures de données utilisées pour transférer des données entre les couches, en structurant et validant les données circulantes entre la couche Application et les couches externes.

# COUCHE DE PRÉSENTATION

- La couche de présentation sert de pont entre le monde extérieur (comme les clients HTTP, l'interface utilisateur ou les API externes) et la logique principale de l'application.
- Adapte les données entrantes (requêtes HTTP) en modèles d'application internes.
- Adapte les données sortantes aux formats appropriés (réponses JSON).
- Gère les validations au niveau de l'API (par exemple, la validation du modèle via les annotations de données).
- Gère les problèmes de sécurité au niveau de l'API (authentification, autorisation).

contient :

✓ Les Contrôleurs API : gèrent les requêtes HTTP, valident les entrées, invoquent les services applicatifs et renvoient les réponses appropriées.

# PROJET EXEMPLE (GESTION DES PRODUITS)

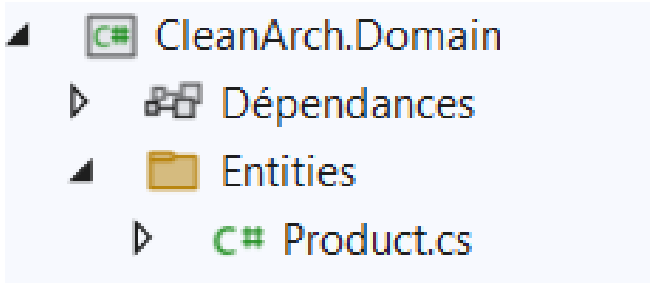
- Nous allons construire un système de gestion de produits simple utilisant les principes de Clean Architecture avec l'API Web ASP.NET Core, l'approche Code-First d'Entity Framework Core (EF Core) et SQL Server comme base de données.
- Commençons par créer une solution vide nommée ProductManagment.
- Ajouter à la solution un projet de type bibliothèque de classes nommé DomainLayer.
- La couche Domaine définit les entités et règles métiers essentiels, indépendamment des Framework ou de l'infrastructure.
- Dans le projet DomainLayer ajouter un dossier nommé « Entities ».
- Dans le dossier Entities ajouter un fichier de classe nommé « Product.cs »
- Ajouter le code qui suit à cette classe :

# CRÉATION DE LA COUCHE DE DOMAINE

```
public class Product
{
        public int Id { get; set; }
        [Required, StringLength(100, MinimumLength = 2)]
        public string Name { get; set; } = null!;
        [StringLength(500)]
        public string? Description { get; set; }
        [Column(TypeName = "decimal(18,2)")]
        public decimal Price { get; set; }
        [Range(0, int.MaxValue, ErrorMessage = "Stock cannot be negative.")]
        public int Stock { get; set; }

        // Business rule: Ensure Price is positive and Stock is not negative
        public void ValidateBusinessRules()
        {
            if (Price <= 0)
                throw new InvalidOperationException("Product price must be positive.");
            if (Stock < 0)
                throw new InvalidOperationException("Product stock cannot be negative.");
        }

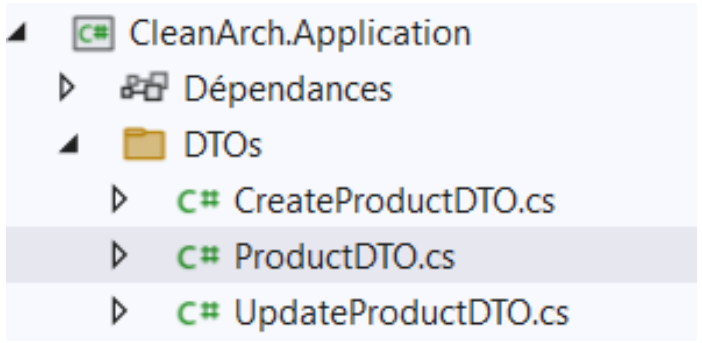
}
```



# LA COUCHE APPLICATION

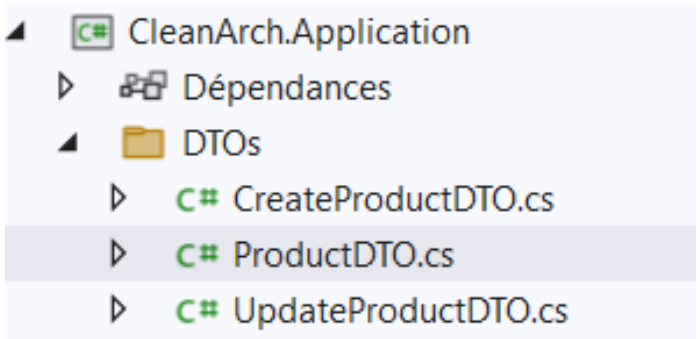
- Ajouter un autre projet de type bibliothèque de classes nommée ApplicationLayer.
- Commençons par créer  les classes DTO
- Créer un nouveau dossier nommé DTOs.
- Créez un fichier de classe nommé ProductDTO.cs dans ce dossier

```
public class ProductDTO
{
    public int Id { get; set; }
    public string Name { get; set; } = null!;
    public string? Description { get; set; }
    public decimal Price { get; set; }
    public int Stock { get; set; }
}
```



# LA COUCHE APPLICATION

- Sous le dossier DTOs, Ajouter la classe CreateProductDTO.CS

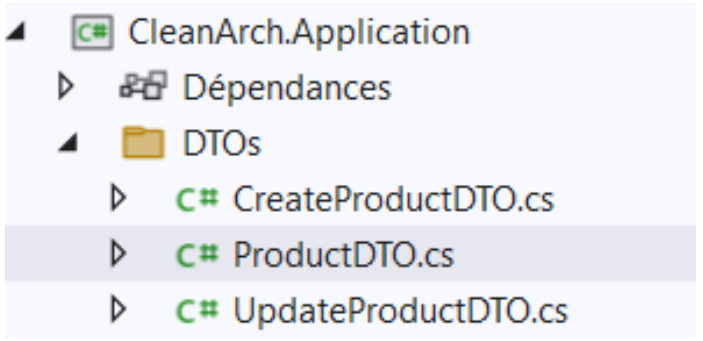



```
public class CreateProductDTO
{
    [Required(ErrorMessage = "Product name is required.")]
    [MaxLength(100, ErrorMessage = "Product name cannot exceed 100 characters.")]
    public string Name { get; set; } = null!;
    [StringLength(500)]
    public string? Description { get; set; }
    [Range(0.01, 1000000, ErrorMessage = "Price must be positive and less than 1000000.")]
    public decimal Price { get; set; }
    [Range(0, int.MaxValue, ErrorMessage = "Stock cannot be negative.")]
    public int Stock { get; set; }

}
```

# LA COUCHE APPLICATION

- Sous le dossier DTOs, Ajouter la classe **UpdateProductDTO.cs**



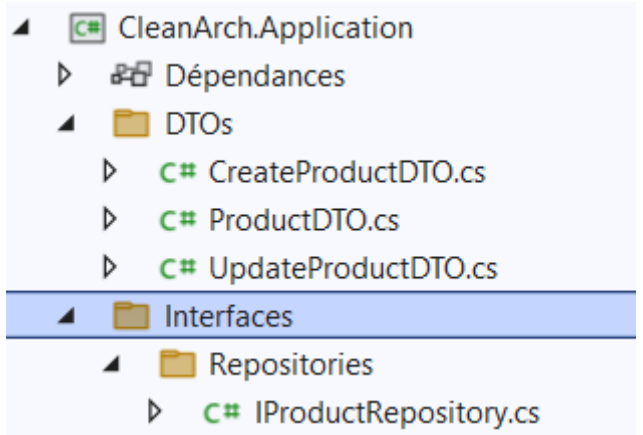

```
public class UpdateProductDTO
{
    [Required]
    public int Id { get; set; }
    [Required(ErrorMessage = "Product name is required.")]
    [MaxLength(100, ErrorMessage = "Product name cannot exceed 100 characters.")]
    public string Name { get; set; } = null!;
    [StringLength(500)]
    public string? Description { get; set; }
    [Range(0.01, 1000000, ErrorMessage = "Price must be positive and less than 1000000.")]
    public decimal Price { get; set; }
    [Range(0, int.MaxValue, ErrorMessage = "Stock cannot be negative.")]
    public int Stock { get; set; }
}
```

# LA COUCHE APPLICATION- REPOSITORY INTERFACES

- Dans la couche application, ajouter un dossier nommé « Interfaces ».
- Sous le dossier « Interfaces » ajouter un nouveau dossier nommé « Repositories ».
- Dans le dossier « Repositories », créez une interface nommée IProductRepository.cs

```
public interface IProductRepository
{
    Task<IEnumerable<Product>> GetAllAsync();
    Task<Product?> GetByIdAsync(int id);
    Task<Product?> AddAsync(Product product);
    Task UpdateAsync(Product product);
    Task DeleteAsync(int id);

}
```

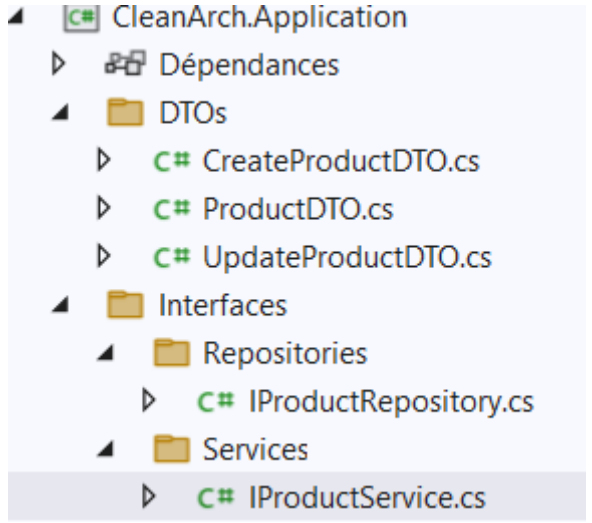


# LA COUCHE APPLICATION- PRODUCT SERVICE INTERFACE

- Dans la couche application, et sous le dossier « Interfaces » ajouter un nouveau dossier nommé « Services ».
- Dans le dossier « Services », créez une interface nommée **IProductService.cs**

```
public interface IProductService
{
    Task<IEnumerable<ProductDTO>> GetAllProductsAsync();
    Task<ProductDTO?> GetProductByIdAsync(int id);
    Task<ProductDTO?> AddProductAsync(CreateProductDTO productDto);
    Task UpdateProductAsync(UpdateProductDTO productDto);
    Task DeleteProductAsync(int id);

}
```



# C. APPLICATION- PRODUCT SERVICE IMPLEMENTATION

- Dans la couche application, ajouter un nouveau dossier nommé « Services ».
- Dans le dossier « Services », créez un fichier de classe nommé **ProductService.cs**

```
public class ProductService : IProductService
{
    private readonly IProductRepository _repository;
    public ProductService(IProductRepository repository)
    {
        _repository = repository;
    }
    private ProductDTO MapToDTO(Product product)
    {
        return new ProductDTO
        {
            Id = product.Id,
            Name = product.Name,
            Description = product.Description,
            Price = product.Price,
            Stock = product.Stock
        };
    }
```

# C. APPLICATION- PRODUCT SERVICE IMPLEMENTATION

```
public async Task<IEnumerable<ProductDTO>> GetAllProductsAsync()
{
    var products = await _repository.GetAllAsync();
    var dtos = new List<ProductDTO>();
    foreach (var product in products)
    {
        dtos.Add(MapToDTO(product));
    }
    return dtos;
}
public async Task<ProductDTO?> GetProductByIdAsync(int id)
{
    var product = await _repository.GetByIdAsync(id);
    if (product == null)
        return null;
    return MapToDTO(product);
}
```

# C. APPLICATION- PRODUCT SERVICE IMPLEMENTATION

```
public async Task<ProductDTO?> AddProductAsync(CreateProductDTO productDto)
        {
            var product = new Product
            {
                Name = productDto.Name,
                Description = productDto.Description,
                Price = productDto.Price,
                Stock = productDto.Stock
            };
            product.ValidateBusinessRules();
            var newProduct = await _repository.AddAsync(product);
            if (newProduct != null)
                return MapToDTO(newProduct);
            return null;
        }
        public async Task UpdateProductAsync(UpdateProductDTO productDto)
        {
            var product = await _repository.GetByIdAsync(productDto.Id);
            if (product == null) throw new Exception("Product not found.");
            product.Name = productDto.Name;
            product.Description = productDto.Description;
            product.Price = productDto.Price;
            product.Stock = productDto.Stock;
            product.ValidateBusinessRules();
            await _repository.UpdateAsync(product);
        }
```

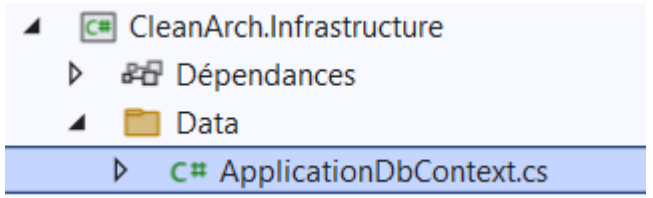
```
        public async Task DeleteProductAsync(int id)
            {
                await _repository.DeleteAsync(id);
            }
    }
}
```

# LA COUCHE INFRASTRUCTURE

- Sous notre solution ProductManagment Créer un nouveau projet de type bibliothèque de classes nommé « InfrastructureLayer ».
- Installer le package Microsoft.EntityFrameworkCore.SqlServer
- Installer le package Microsoft.EntityFrameworkCore.Tools
- Créez un nouveau dossier nommé « Data » dans le projet Infrastructure . Ensuite, dans le dossier Data , créez un fichier de classe nommé ApplicationDbContext.cs

# LA COUCHE INFRASTRUCTURE

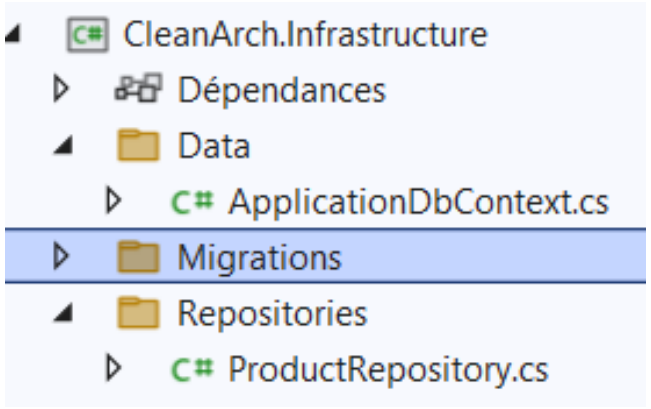
```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        // Seed initial products
        modelBuilder.Entity<Product>().HasData(
            new Product { Id = 1, Name = "Laptop", Description = "High-end gaming laptop", Price
= 1500.00m, Stock = 10 },
            new Product { Id = 2, Name = "Smartphone", Description = "Latest model smartphone",
Price = 800.00m, Stock = 25 },
            new Product { Id = 3, Name = "Wireless Headphones", Description = "Noise cancelling
headphones", Price = 200.00m, Stock = 40 }
        );
    }
    public DbSet<Product> Products { get; set; } = null!;
}
```



# LA COUCHE INFRASTRUCTURE

- Dans le projet infrastructure, ajouter un dossier nommé « Repositories », dans ce dossier créer la classe **ProductRepository.cs**

```
public class ProductRepository : IProductRepository
{
    private readonly ApplicationDbContext _context;
    public ProductRepository(ApplicationDbContext context)
    {
        _context = context;
    }
    public async Task<IEnumerable<Product>> GetAllAsync()
    {
        return await _context.Products.AsNoTracking().ToListAsync();
    }
    public async Task<Product?> GetByIdAsync(int id)
    {
        return await _context.Products.AsNoTracking().FirstOrDefaultAsync(prd => prd.Id == id);
    }
    public async Task<Product?> AddAsync(Product product)
    {
        await _context.Products.AddAsync(product);
        await _context.SaveChangesAsync();
        return product;
    }
```



# LA COUCHE INFRASTRUCTURE

```
 public async Task UpdateAsync(Product product)
    {
        _context.Products.Update(product);
        await _context.SaveChangesAsync();
    }
    public async Task DeleteAsync(int id)
    {
        var product = await _context.Products.FindAsync(id);
        if (product != null)
        {
            _context.Products.Remove(product);
            await _context.SaveChangesAsync();
        }
    }
}
```

# LA COUCHE PRÉSENTATION – WEB API LAYER

- Dans la même solution, créer un nouveau projet de type ASP.NET Core WEB API nommé « ProductsAPI ».
- Installer le package Microsoft.EntityFrameworkCore.SqlServer
- Dans le dossier Controllers ajouter un nouveau contrôleur API vide nommé ProductController.cs
-

# LA COUCHE PRÉSENTATION – WEB API LAYER

```
[Route("api/[controller]")]
[ApiController]
public class ProductController : ControllerBase
{
    private readonly IProductService _service;
    public ProductController(IProductService service)
    {
        _service = service;
    }
    [HttpGet]
    public async Task<IEnumerable<ProductDTO>> Get()
    {
        return await _service.GetAllProductsAsync();
    }
    [HttpGet("{id}")]
    public async Task<ActionResult<ProductDTO>> Get(int id)
    {
        var product = await _service.GetProductByIdAsync(id);
        if (product == null)
            return NotFound();
        return Ok(product);
    }
```

# LA COUCHE PRÉSENTATION – WEB API LAYER

```
 [HttpPost]
    public async Task<IActionResult> Post(CreateProductDTO productDto)
    {
        var product = await _service.AddProductAsync(productDto);
        return Ok(product);
    }
    [HttpPut("{id}")]
    public async Task<IActionResult> Put(UpdateProductDTO productDto)
    {
        await _service.UpdateProductAsync(productDto);
        return NoContent();
    }
    [HttpDelete("{id}")]
    public async Task<IActionResult> Delete(int id)
    {
        await _service.DeleteProductAsync(id);
        return NoContent();
    }
}
```

# LA COUCHE PRÉSENTATION – WEB API LAYER

**appsettings.json File**

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=ProductManagementDB;Trusted_Connection=True"
  }
}
```

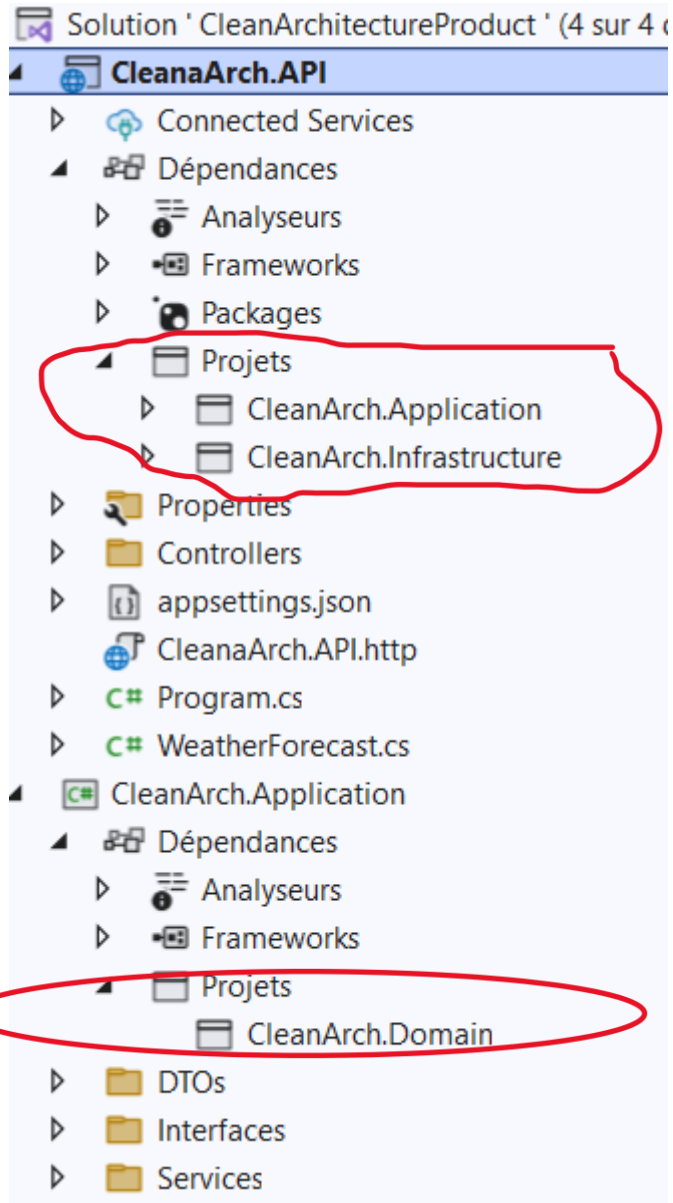
# LA COUCHE PRÉSENTATION – WEB API LAYER

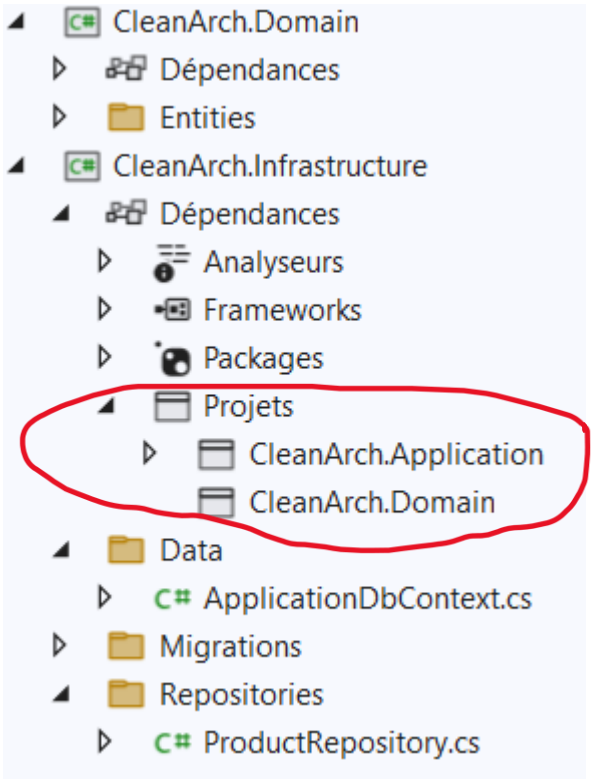
**Program.cs File**

```
// Add DbContext with SQL Server
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// Register services and repositories
builder.Services.AddScoped<IProductRepository, ProductRepository>();
builder.Services.AddScoped<IProductService, ProductService>();
```

# LES REFERENCES DE PROJETS ENTRE LES DIFFERENTES COUCHES



# LA COUCHE PRÉSENTATION – WEB API LAYER

## Migration Entity Framework et création de la base de données SQL SERVER

- Dans les propriétés de la solution choisir le projet API comme projet de démarrage de l'application.
- Lancer le package manager console
- Choisir le projet Infrastructure comme projet cible de la migration (puisqu'il contient la classe de contexte).
- Lancer les commandes :

  add-migration dbcreate

  update-database

- Exécuter l'application et tester les Endpoints avec swagger.

